

Practical Applitudes: Case Studies of Applications of the ZigZag Hypermedia System

Adam Moore, James Goulding, Tim Brailsford & Helen Ashman

School of Computer Science and Information Technology

Jubilee Campus, University of Nottingham

Nottingham, NG8 1BB, United Kingdom

{axm,jog,tjb,hla}@cs.nott.ac.uk

ABSTRACT

ZigZag is a paradigm of hypermedia that consists of a multidimensional system of principled interconnections. Its basic features and specifications are now well known, but despite this, very few practical applications have been described or discussed. This paper examines two projects as case studies. These projects both use the unique properties of ZigZag in order to solve real-world problems. One of these case studies is a personal information management system for mobile phones, and the other is a bioinformatics visualization system. Although superficially extremely different, these areas both make use of information that is loosely structured and deeply interconnected.

Categories and Subject Descriptors

H.5.4 [Information Interfaces and Presentation]:

Hypertext/Hypermedia – *Architectures, Navigation, Theory, User Issues*

General Terms

Design, Human Factors.

Keywords

ZigZag, Application, Applitude, Case Studies, PIM, Bioinformatics.

1. INTRODUCTION

The ZigZag system is a hypertextual environment invented by Ted Nelson, that embodies a multidimensional lattice of principled interconnections [5,6,9]. Information in ZigZag is contained in cells that can be connected along an unlimited number of intersecting dimensions. One way to understand ZigZag is to think of it as a multi-dimensional spreadsheet that is capable of representing structures more complex than a simple grid. The basic features and specifications have been discussed in the hypertext community for some time now [5,6], but despite this

descriptions of practical applications are rare [5,8]. Currently, a simple personal scheduler and budget manager exists, and most demonstrations are accompanied by a representation of genealogical data. However, the paucity of practical applications often leads to the question of “what can I actually do with it?”. In this paper we shall address this issue by examining case studies of two prototype systems that are designed to use the unique features of the ZigZag paradigm to solve real-world problems. Although these systems are still at the prototype stage of development, they are ongoing projects that are aimed at very practical problems, and they are already sufficiently mature to illustrate the broad applicability of ZigZag.

2. OVERVIEW OF ZIGZAG

Throughout this paper we will refer to the constructs and core functions of ZigZag. The following section therefore introduces these concepts.

At its core, ZigZag can be thought of as a space filled with cells. These cells can contain any *one* type of data (text, graphics, sound, video, etc.). They are connected to each other along dimensions, the positive side of one cell (named by Nelson the posward side) connecting to the negative side of another (named by Nelson the negward side). For any dimension, a cell can only have one connection in the posward direction, and one in the negward direction. It should be mentioned here that although the above law implies a directionality to ZigZag dimensional connections, the direction may not necessarily have any semantic meaning in any space. These connections along a dimension are not typed, and are better thought of as adjacencies – there are no typed links in ZigZag. All aspects of ZigZag are themselves encoded in ZigZag, with all information being stored in cells (the cell being the sole primitive unit of ZigZag existence). Hence dimensions are themselves cells, and the connections merely represent the ordering of cells along any given dimension. This ensures that all paths are non-branching, and thus embodies the simplest possible mechanism for traversing links.

Dimensions in ZigZag can have any name. However, there is a convention that these names start with a single character letter code, followed by a full stop, and then the name of the dimension. Current character codes used include g for graphics-related dimensions, v for view-related dimensions and general dimensions are usually coded with a d. Three utility dimensions exist in most current implementations of ZigZag, d.1, d.2 and d.3. These can be thought of as analogous to sheets of paper, with d.1 and d.2 being the length and depth of a sheet, and d.3 providing a new sheet.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

HT'04, August 9–13, 2004, Santa Cruz, California, USA.

Copyright 2004 ACM 1-58113-848-2/04/0008...\$5.00.

Clones are also an important part of ZigZag. They represent an implementation of transclusion at the cell level. This means that the same cell contents can have multiple references at the same time.

3. APPLITUDES, NOT APPLICATIONS

In the majority of the computing world, a paradigm at the core of most user interface design is that of the *application*. These are self-contained programs, dedicated to a particular task or group of related tasks. Applications operate upon stored information, usually in a format that is tailored to that specific information. This can have the unfortunate side effect that software from different vendors performing the same function can often have difficulty accessing what is essentially the same information. In practice the tasks that people undertake with computers often overlap between the tasks performed by several different applications. This is what can be called the “application prison”[3], and it makes performing those tasks unwieldy at best. This limitation of system design is wholly avoided by the ZigZag approach, in which there are no isolated applications.

The core tenet of ZigZag is that all information is complex and interlinked – as Ted Nelson wrote in his seminal book *Computer Lib* “everything is deeply intertwined” [4]. At the most fundamental level this has two implications:

- Potentially, every piece of information stored in ZigZag may be connected with every other piece of information. As such, ZigZag may be viewed as a hypertext system.
- There are no separate applications in ZigZag – all information is therefore inherently programmatical, that is, every piece of information can be addressed for any context.

Any information structure following the rules of the ZigZag universe is termed a *zzStructure*, and a *zzStructure* that is designed to perform a particular task is named by Nelson an *applitude*. While there are many parallels between an *applitude* and an *application*, there are also fundamental differences. Unlike applications an *applitude*’s core components can easily be reused either by means of cloning (i.e. transclusion) or by connecting them along different dimensions. An *applitude* is not merely an application that has been designed to work with ZigZag data, but it is rather a part of a *zzStructure*, utilizing a set of views and dimensions in order to express a specific functionality over a particular part of that space. It is a zone of purpose, populated by cells that are specific to that purpose. Because an *applitude* is itself a *zzStructure*, in ZigZag functional software and data are different aspects of the same thing. Therefore, all *applitudes* are data, and all data structures are *applitudes*. This reflects Nelson’s prime design paradigm of ZigZag that it should be “Turtles all the way down”¹. The main control functions of any implementation of ZigZag are themselves a part of the *zzStructure*. There are some similarities with Lisp, in that ZigZag dimensions are analogous to lists and cells analogous to atoms. However, there are also fundamental differences, in that Lisp structures tend to be based upon recursion, whereas all states in ZigZag are themselves stored in a *zzStructure*. It is

possible to subsume the functionalities of Lisp with *zzStructure*, but it is not necessary to utilise Lisp processing paradigms within ZigZag. In many respects ZigZag represents the ultimate open data format, where the functionality of the software is implicit in the data itself, therefore **there are no applications**, and interoperability is not an issue.

There are certain similarities between ZigZag and some other systems, in particular those concerned with storage – such as XML and relational database technologies. A detailed comparison is outside the scope of this paper, but we will briefly consider some of the most important issues. XML technologies are often cited as an alternative to ZigZag - with claims ranging from the assertion that *zzStructures* could be expressed in XML, to the fact that *applitudes* provide equivalent functionality to web services. There are various similarities between ZigZag and XML, but there are also some substantial differences. It is possible for *zzStructures* to subsume the hierarchical representation of XML [2], although they do not have any strict rules for validation. The fundamental difference is that ZigZag operates at a different level of abstraction from either XML or a database, which is a storage layer, ZigZag itself is an abstract machine. With respect to relational databases, it is quite possible to store the information contained in ZigZag cells within a database, but the connections would then be far more difficult to maintain – because the relational database paradigm does not provide the syntax or semantics for manipulating orthogonally intersecting dimensions. There is currently no method for expressing the equivalent of the schema of a database (i.e. a strict set of rules for structure) in ZigZag. It is arguable as to whether such a tool is useful, or even possible, in *zzStructures*, but such arguments are beyond the scope of the current discussion.

A major human factors issue with the current set of tools designed to interact with *zzStructures* is that they are heavily biased towards spatial representations and keyboard/pen interfaces. This implies that those users less comfortable with graphical user interfaces and more biased towards textual/verbal representations of information (i.e. verbalisers rather than visualisers) are likely to be disadvantaged. This problem is intrinsic to the current generation of views that directly visualize the connections. In the future, a simple ZigZag API will allow software to interact with *zzStructures* at the abstract level of cells and connections, thus allowing views to be constructed that are less dependent upon spatial cognition. This API is currently being developed by the authors in collaboration with Ted Nelson and other members of the ZigZag project. In addition to this, experiments are being conducted with alternative control mechanisms for the current spaces, such as gamepad controllers and interactive whiteboards. In the future these are likely to provide the primary user interface for many ZigZag systems.

4. CASE STUDIES

In this paper we examine the practical applicability of the ZigZag paradigm to real-world problems, by examining case studies of *applitudes* in two different implementations. These examples were deliberately chosen to be extremely different from each other in order to illustrate the generalizable nature of the ZigZag approach to information management.

The first of these case studies is *zzPhone* – a prototype information manager for mobile phones that is currently under

¹ For a brief discussion of the origins of this phrase see, for example:
<http://www.apatheticagnostic.com/Essays/meds/med09/med173.html>

active development for the Sony-Ericsson P800 phone, using the *Symbian* operating system² with the UIQ pen-based user interface³. *zzPhone* is inspired by the Personal Information Manager created by Marlene Mallicoat and described in [5].

The second case study is that of a *Bioinformatics Workbench* – essentially an information manager for bioinformatics. Bioinformatics data is notoriously complex, and managing it using conventional approaches can often be an extremely difficult task [3]. A prototype *Bioinformatics Workbench* is currently undergoing development using GZigZag, an open source implementation of ZigZag [1].

5. CASE STUDY 1: *zzPhone*

Typically modern cell phones contain hundreds, if not thousands of items of information, almost all of which are related to each other. Contacts are associated categorically as well as alphanumerically, and also directly to individual phone numbers, which are related to each other in terms of area codes. These phone numbers are related to both individual phone conversations and SMS texts, and both of these separate events *themselves* occur on a timeline and so relate to each other temporally. Contacts also have addresses, which relate to each other both alphanumerically and geographically. The intertwined relationships of all of this information are both extensive and massively complex, and yet in the current generations of phones this information is stored using a simplistic model. Where an underlying connectivity should be flourishing, it is instead severely constrained. The conventional phone menu system, with its brittle close-ended hierarchy, could not possibly hope to represent, or indeed cope with, this plethora of relationships.

The ZigZag phone applitude described here imposes an orthogonal structure upon the system, and thus enforces the channelling of information. These channels – the ZigZag dimensions – can either be tuned in or out of as necessary, in order to provide the clarity a human requires to find a path from one point of information to another. That is, by selecting the dimensions of interest connecting the data, it can be seen in the context desired. By representing a cell phone's data as a *zzStructure*, we are providing a navigational tool that allows a user to manage the entanglement of connections and relationships that are discarded by traditional systems.

5.1 The *zzPhone* User Interface

Mobile devices necessarily have highly constrained user interfaces in comparison with desktop computers, and in particular they have an extremely restricted screen size and resolution. The current viewer design for *zzPhone* is shown in Figure 1.

All of the cells contain one item of multimedia information, and they are rendered in a grid like structure. The current focus cell is rendered in the centre of the screen, with the display algorithm recursing out from this point to produce the perspective visualisation. Navigation of the structure is achieved through a single-click on the body of a cell – at which point animated transitions bring it to the centre of the screen and it becomes the system's new focus.

The view shown in Figure 1 allows a maximum of 33 cells to be rendered on screen, through the partial occlusion of several

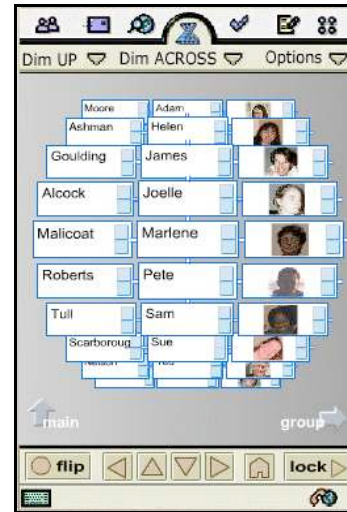


Figure 1. The *zzPhone* Viewer showing the *Contacts*, *Firstname* and *Photo* ranks.

of these, with their full contents often being partially hidden. However, partially occluded cells may still be selected and therefore navigational integrity is maintained. The vertical and horizontal arrows in the lower corners of the display indicate the current active dimensions that the user is tuned into, with the orientation of the arrow indicating the direction in which that dimension is being represented.

Dimension changes are controlled from the menu bar at the top of the screen, allowing users to select the channels of information which are visually represented. The *dim ACROSS* and *dim UP* menus allow the user to set the current x and y dimensions respectively by selecting from a list of available dimensions. By default, only those dimensions in which the focus cell currently has connections are displayed (these dimensions usually being the most relevant to the user's current task). This behaviour may be changed via a setting in the *Options* menu, so that all dimensions are made available. The button bar towards the bottom of the screen provides: four *navigational arrows*; a *flip* button which transposes the viewing axes; a *home* button which moves the focus to cell zero and resets the dimensions to their starting states; and finally the view *lock* button which alternates between the row and column views of the structure [6].

5.2 The *zzPhone* Information Structure

The starting point for the *zzPhone* is its primary rank, a row of *zzCells* extending down from the home cell and connected in dimension *d.main*. This primary rank makes up the *zzPhone*'s main menu, and includes *contacts*, *numbers*, *photos*, *cities*, *calls* and *texts*. Just as in a conventional menu, each of these choices is a link to the sub list represented by the parent menu item. Bringing the cursor to focus upon any menu item will reveal its respective sub list and this is illustrated in Figures 2 and 3, where the system is focused upon the contacts and cities menu cells respectively.

Standards in the use of dimensions are key to providing the user with a stable map of the *zzPhone*'s underlying structure. Outside of the menu rank, *d.main* represents a strong alphanumeric relationship throughout, and cells connected within it will consistently appear as a list arranged in alphabetical order. This is particularly useful for navigation

² <http://www.symbian.com/>

³ <http://www.uiq.com/>

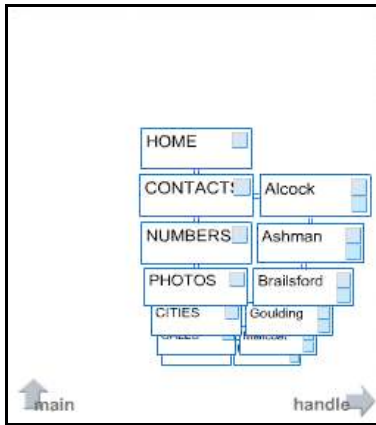


Figure 2. The *zzPhone* main menu, with the contact cell as focus – and thus the contact menu expanded.

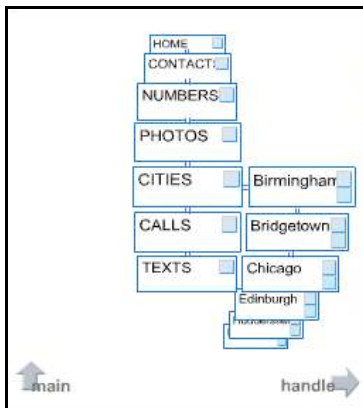


Figure 3. The *zzPhone* main menu, with the cities cell as focus – and thus the cities menu expanded.

and the consistency with which *d.main* is employed is integral to the user as a fallback point in traversing the structure. In contrast the convention of *d.handle* is to provide connections that serve as convenient entrance points to the data and *zzStructure* proper, and it also performs this task in a consistent fashion. The final vital convention utilized within the *zzPhone* aplitude is that of *d.group*, which provides connections to sets of cells which are linked conceptually in order to provide a grouping rank.

5.3 Examples of Navigation Using *zzPhone*

Consider one of the most common issues in saving numbers on a cell phone – where exactly should a new contact number be saved? If John Doe provides his work and home numbers, should they be saved as John Doe Work and John Doe Home, so that they are easily accessible via one click from the main contact list, or saved under a single John Doe contact with both his numbers being put into the contact's sub categories? There is, of course, no correct answer to this dilemma, and one has to make an instant, arbitrary decision. From that point onwards, use of those numbers are constrained one way or the other, due to the limitations of the conventional phone's hierarchical storage system. In the *zzPhone* aplitude, however, a phone number exists in many different contexts

simultaneously. For example, it may be found in an alphanumerical list of all the available numbers, a list of specific work numbers or a list of numbers pertaining to a single contact. The number is not copied into each of these lists but rather it exists in all of them at once through transclusion. From a *phone number* cell we can view any of these contexts instantly by means of a simple drop down menu.

In addition to this, consider what happens if Jane Doe now supplies her work and home numbers, but the latter turns out to be the same as John's, which has previously been stored? However a conventional phone system is organised, it is impossible to represent this relationship, and there is no way to make the fact that both persons are contactable at the same number location. In *zzPhone*, however, it is possible to switch to the *owner* dimension and see a list of all people available at a certain location. In addition, a single phone number may have multiple owners, with each of those owners simultaneously having multiple numbers. It is impossible in any hierarchical system to model a multi-attributed circular relationship of this nature.

To illustrate this, Figure 4 shows a display containing all those numbers owned by a single contact, along with the other details adjacent to them in *d.group*. Changing the vertical dimension to *d.owner* tunes the display into those people who share that specific work number, as is shown in Figure 5.

Figure 6 illustrates how by clicking on the "mobile" cloned cell from Figure 4, and then switching the dimensions via the flip button, a list of contacts who have mobile numbers can be shown. This is an emergent property of the *zzStructure* - there is no need for complications such as SQL queries and this is particularly valuable on a mobile device where computing resources are usually extremely limited. As a final *zzPhone* example, imagine that the person being called wasn't in, and that it was important to find someone who lives nearby (i.e. in the *same city*). This can be achieved by changing the horizontal dimension shown in Figure 4 to *d.address*, selecting the city address, and then switching back to *d.clone*. The results of such a procedure are shown in Figure 7, where a list of other contacts who also live in Nottingham is shown who may alternatively be contacted.

These examples highlight the flexibility of the system. Anything that is stored as a relationship can be accessed with minimal effort. There is no complex search language required to facilitate these results and no informational relationships are ignored to force intertwined data into an inappropriate hierarchical menu system. It is possible to move rapidly to any specific information item through movement of the cursor cell and tuning in and out information channels that are of interest. It is also possible to browse the *zzStructure*, in order to uncover previously unknown relationships, this discovery of emergent properties being a particularly relevant feature for the bioinformatics workspace.

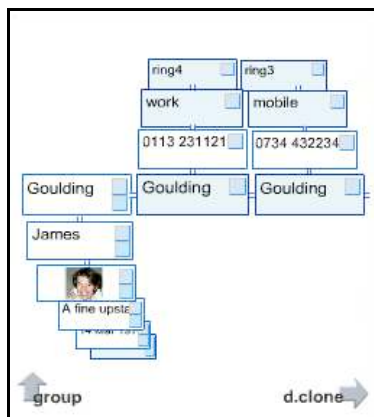


Figure 4. *zzPhone* showing all of the numbers that James Goulding owns (NB the vertical dimension is *d.group*)

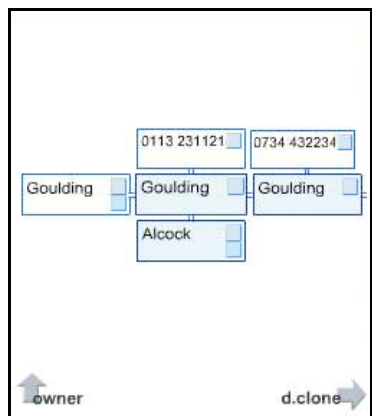


Figure 5. *zzPhone* showing the people who share a work number with James Goulding (NB the vertical dimension has been changed to *d.owner*).

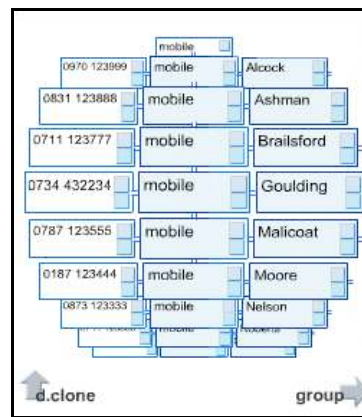


Figure 6. *zzPhone* showing those contacts who have mobile phone numbers.

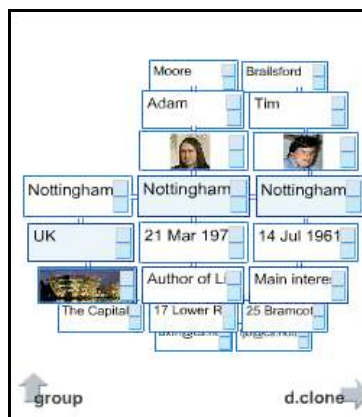


Figure 7. *zzPhone* showing other contacts who also live in Nottingham.

5.4 Adding a New Entry

Currently the *zzPhone* is functional solely as a viewer, and is limited to browsing the available structure. It serves as an illustrative test of the potential capabilities of ZigZag for personal information management (note, however, that the aptitude can be fully manipulated within GZZ). At present adding a new entry into the structure – whether that be a contact, a phone number or indeed any entity – involves hand crafting of the necessary cells and linking them along the appropriate dimensions, via the engine's API calls. Automating this process should be a simple matter, but one that is crucial to the system's usability; it is envisioned that in the future an entry will be added by selecting an appropriate template and filling in the required values. These templates will be derived from data organised for the entity under consideration along *d.group* and will place the entered information into cells, arranging them via sensible default connections. Similarly, editing of individual cells through the phone's standard input mechanisms is essential. This is a key area for further development, as the ZigZag design paradigm theoretically promises to offer notable benefits for the maintenance and updating of information. It allows the user to

focus upon specific dimensions and hence the current context of use, without being distracted by extraneous information.

6. CASE STUDY 2:

BIOINFORMATICS WORKSPACE

Bioinformatics is a discipline that has been formed by the intersection of the fields of computer science and biology. This has come about because in recent years the technology of biology – particularly in the biomolecular sciences – has become sufficiently advanced that datasets of unparalleled size and complexity are being gathered. Bioinformatics is all about applying techniques of computer science to the problems of storing, retrieving and interpreting such data.

Bioinformatics thus has peculiar problems that are, at least in part, the very reason for the field's existence. Bioinformatics seeks to find answers to the big questions of disease – why do we get ill? how can we prevent disease? and how can we find cures? In order to do this, it is necessary to deal with uniquely complex, diverse and deeply interrelated sets of information. This information ranges from patient records, via the output of diagnostic tests, to the results of experiments, both from laboratory benches and from computational simulation.

The demand on infrastructures to deal with this rich information set is, therefore, to provide storage and query mechanisms that can simply, completely and intuitively represent all these different pieces of data and allow the construction of analytical workflow and higher-order organization from all the disparate pieces. The usual approach to this is to use very large amounts of processing power to extract information. The idea of using ZigZag in the *Bioinformatics Workspace* is that ZigZag's ability to allow channels of information to be *tuned* in and out could extend the capabilities of humans to browse this information and expose new relationships in the data. The hope is that this will ultimately leverage human intellect, rather than processor cycles.

6.1 Main Street

The first view that is usually seen is the top level organization of the information to be used. This highest organizing list in an applitude is usually called *Main Street*, the place where all other zones lead off – an example of this is given in Figure 8.

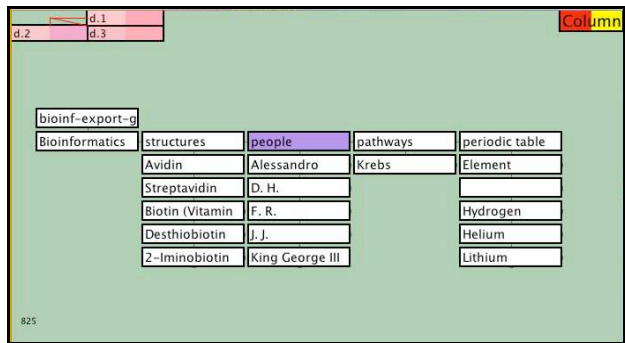


Figure 8. The *Bioinformatics Workbench*: Main Street.

This uses one of the main conventions of ZigZag, the list with a head cell. Here, Bioinformatics is split into four main categories: Structures, People, Metabolic Pathways, and The Periodic Table. The head cell for a *zzList* is the left-most in a horizontal list, or the top in a vertical list. Hence, Main Street is a horizontal list of head cells. Each category is described in more detail in the following sections.

In the system described here, we have implemented some simple biological systems in the *Bioinformatics Workspace*. These systems will be familiar to anyone with some knowledge of biology, but they are also intended to be simple enough to be comprehensible to non-biologists. The purpose of this is to demonstrate how ZigZag is capable of representing bioinformatics information – it is likely that systems used by working bioinformaticians would be more complex and rather less accessible to lay readers.

6.2 People

It is useful to have a grasp over which information is produced by which researchers, so the names of people important to the

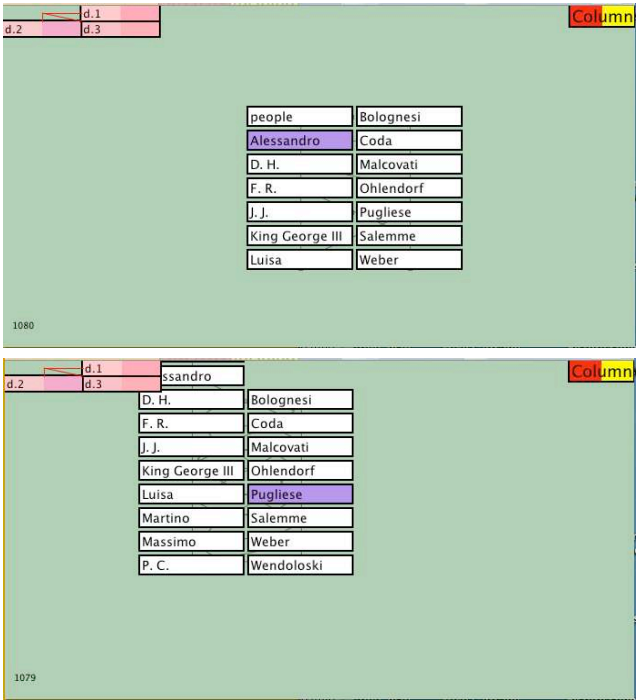


Figure 9. The *Bioinformatics Workbench*: A double alphabetized list of people. Above – the focus cell is in the first name list, and so people are displayed alphabetically by first name. Below – the focus cell is in the second name list and so the people are displayed alphabetically by second name.

field are stored here (see Figure 9). The list is a simple, double-alphabetized structure utilizing separate dimensions for first and second names. This is analogous to the contacts list described for *zzPhone* (see Figure 1). People can, of course, be present in different contexts and these different contexts are represented by the dimensions connecting them to the relevant information. In the current version of the space, people can either be authors of papers (connected along *d.author*, in the order on the publication) or sufferers of a pathology (connected to the disease they suffer from along *d.sufferer*). These relationships will be further illustrated below.

6.3 The Periodic Table

The periodic table is a fundamental piece of information for all forms of chemistry and biochemistry. In its simplest form it is, as its name suggests, merely a table of information about the chemical elements. This view is the first view that can be seen in ZigZag, and it is a 2D table that could easily be implemented in a spreadsheet – with the names of each field at the top, then the units, then the details for each element, starting with the lightest, Hydrogen, then going all the way down to the heaviest (see Figure 10).

Element	Symbol	Atomic mass	Density	Melting point	Boiling point
		g/mol	kg/m ³	K	K
Hydrogen	H	1.01	0.09	14.0	20.4
Helium	He	4.00	0.17	1.0	4.2
Lithium	Li	6.94	534.00	452.0	1590.0
Beryllium	Be	9.01	1800.00	1550.0	3243.0
Boron	B	10.81	2500.00	2600.0	2820.0

Figure 10. The BioInformatics Workbench: A simple table showing properties of the elements.

Of course, there is another view of the periodic table, familiar to most, where elements with similar stoichiometry (i.e. those that partake in the same chemical reactions) are arranged together. In ZigZag this view, as shown in Figure 11, can be created by simply utilizing different dimensions to show the connections, thus the organisation along these dimensions leads to the structure becoming emergent from the list ordering.

Figure 11. The BioInformatics Workbench: A visualization of the periodic table based upon stoichiometry – moving the focus (top to bottom) has the effect of “opening up” the transition elements.

By rotating the x dimension to *d.tablex* and y to *d.ptabley* we can see the classical shape of the table with tall outside *pillars* of the alkali metals (and hydrogen) and the noble gases and long horizontal middle of the transition elements. A single data-set may thus be browsed dynamically to show these different representations, simply by moving the focus and rotating dimensions.

6.4 Structures

The structures and sequences formed by chemical bonds are an extremely important part of bioinformatics information, as from them we can deduce the properties and interactions of the constituent pieces in the processes of life. Within the rank of structural information shown here is data relating to the streptavidin/avidin-biotin system (a well characterized and extremely biologically important system). At the bottom of the table are three chemical structures – those of biotin (Vitamin K) and three of its analogues (structurally similar molecules), desthiobiotin, iminobiotin and diaminobiotin (Figure 12). The chemical structures of these small molecules can be created by joining clones of the appropriate elements from the periodic table together. This has the ramification that the properties of any element and any molecule can be found by following *d.clone* back to the head clone cell, which will be the original entry in the periodic table, from which the appropriate information, for example molecular weight, can be found. By changing the view, the conventional representation of 2D molecules - spheres joined with lines - can be seen (Figure 13). In order to display the similarity of these molecules, equivalent atoms are joined along *d.homology*. Rotating this to the z dimension of the view (as shown in Figure 14), we can move between the four structures and discover that, for example, desthiobiotin is missing the sulphur of the bottom 5-membered ring at the end of the molecule, and iminobiotin has a different configuration of nitrogen rings at the top of the second 5-membered ring.

Above the small molecules are the streptavidin and avidin molecules. These have information stored about the unique IDs (accession numbers) of structures containing these molecules in the publicly available SwissProt and The Protein Data Bank (PDB) databases. Each ID is linked to other key information. For example, the ligand (a small molecule bound to the streptavidin/avidin molecules) in the file is connected along *d.het*. From this we can see that 2ave contains the apo form of avidin, as it is not linked to any ligand, and that 1stp is a file with streptavidin bound to biotin. Rotating to *d.ref*, we can see the references linked to each ID, with the authors linked in from the people rank discussed earlier (Figure 15).

The above description is just the first step in realizing the potential of ZigZag to represent the complex associations of many different types of information. It should be noted, however, that the correct symbology is not used (double lines for double bonds, etc.). This is due to a limitation in the currently implemented views of GZZ, which only allow one dimension to be mapped to each Euclidean axis. It is envisioned that in the future, visualisation of these coplanar sets will be allowed. Then, specialist chemical views will be available to allow representation of such standard conventions. It is also a limitation of these current views that they are all either 2D or pseudo 3D (2D with the addition of limited perspective). New views need to be constructed in order to correctly view 3D, and higher order structures.

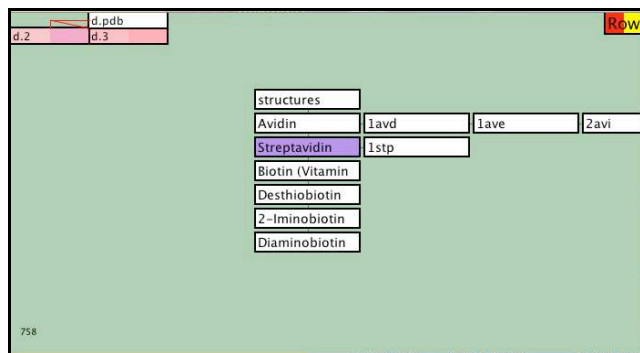


Figure 12. The *BioInformatics Workbench*: Top level representation of the streptavidin/avidin-biotin system with x rotated to d.pdb, showing the Accession IDs of structures containing these molecules.

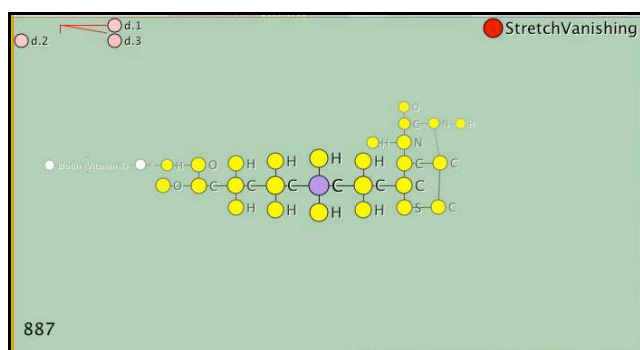


Figure 13. The *BioInformatics Workbench*: The 2-D structure of biotin – created by cloning cells from the periodic table.

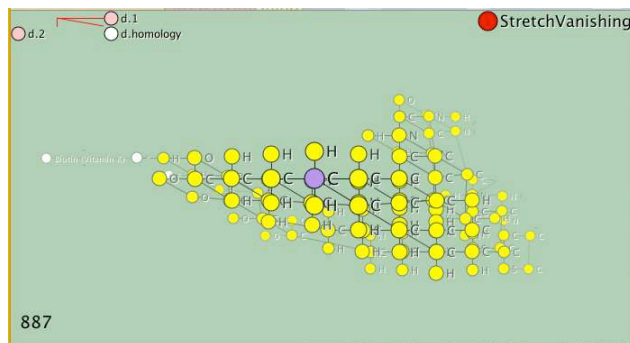


Figure 14. The *BioInformatics Workbench*: The 2-D structure of biotin – showing structural homology with the other small molecules by connections along d.homology (in the z direction).

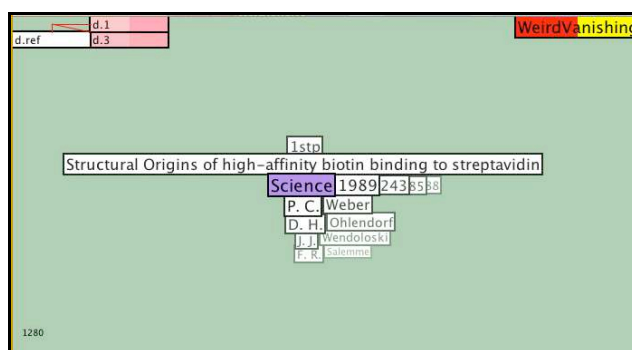


Figure 15. The *BioInformatics Workbench*: The d.ref dimension is used to find bibliographic information.

6.5 Metabolic Pathways

The final part of this space that we wish to discuss is the representation of metabolic pathways. As an example to illustrate this we have chosen the Krebs' (or citric acid) cycle. This is an important part of the body's chemistry – it is the process by which energy is extracted from sugars.

The representation of the pathway starts with glycolysis leading to the production of pyruvate and Acetyl CoA (Figure 16A), which condenses with oxaloacetate to form citrate, which is where the cycle *starts* (Figure 16B). Underneath each molecule in the cycle is the chemical structure of that molecule, again with elements cloned from the periodic table. Where enzymes and other inputs are involved in processes, they are represented below a cell with a star in it, and the results of the reaction are shown above, with the next molecule in the cycle on the right of the star. This star is, in fact, a *relcell* – a cell which implies a relationship between the cells linked to it. Here the cells linked to it along *d.1* are the previous and next molecule in the Krebs' cycle, and the cells linked along *d.2* are the inputs and outputs from the chemical reaction.

A notable point here is that the simple representation of the cycle in ZigZag leads to it inherently having a cyclical structure, that is, its representation immediately follows, and is emergent from, its structure within the space. This is especially useful in biochemistry and bioinformatics in general, where complex processes have many geometries,

which may not be obvious from their representation on paper. Normally, the Krebs' cycle is one of the most difficult concepts to grasp by people new to it – its static representation on a page is counter-intuitive to its fluid nature, and usually this limitation is overcome either partially by providing non-interactive animations (e.g. animated gifs) or spending a lot of time producing multimedia representations. However, using ZigZag, simply representing the cycle in the space allows an immediate understanding and interaction with the information is simply achieved by navigating it.

Rotating the y dimension to *d.in-out* shows some of the other important biochemical molecules either involved in the production of molecules in the cycle, or produced by further reactions with molecules in the cycle (Figure 16C). Most of these molecules have some pathology associated with them, either due to too much or too little being present in the body. For example, if we put the cursor on Porphyrins, above succinyl-coA, and rotate x to *d.disease*, we can see that porphyria is a disease associated with this molecule (Figure 16D). Rotating y to *d.sufferer* we can see that this was a problem in the British royal family, with King George III being a notable victim of this disease (Figure 16E). He is, of course, in the people rank, off Main Street. This illustrates a key facet of the flexibility and power of ZigZag.

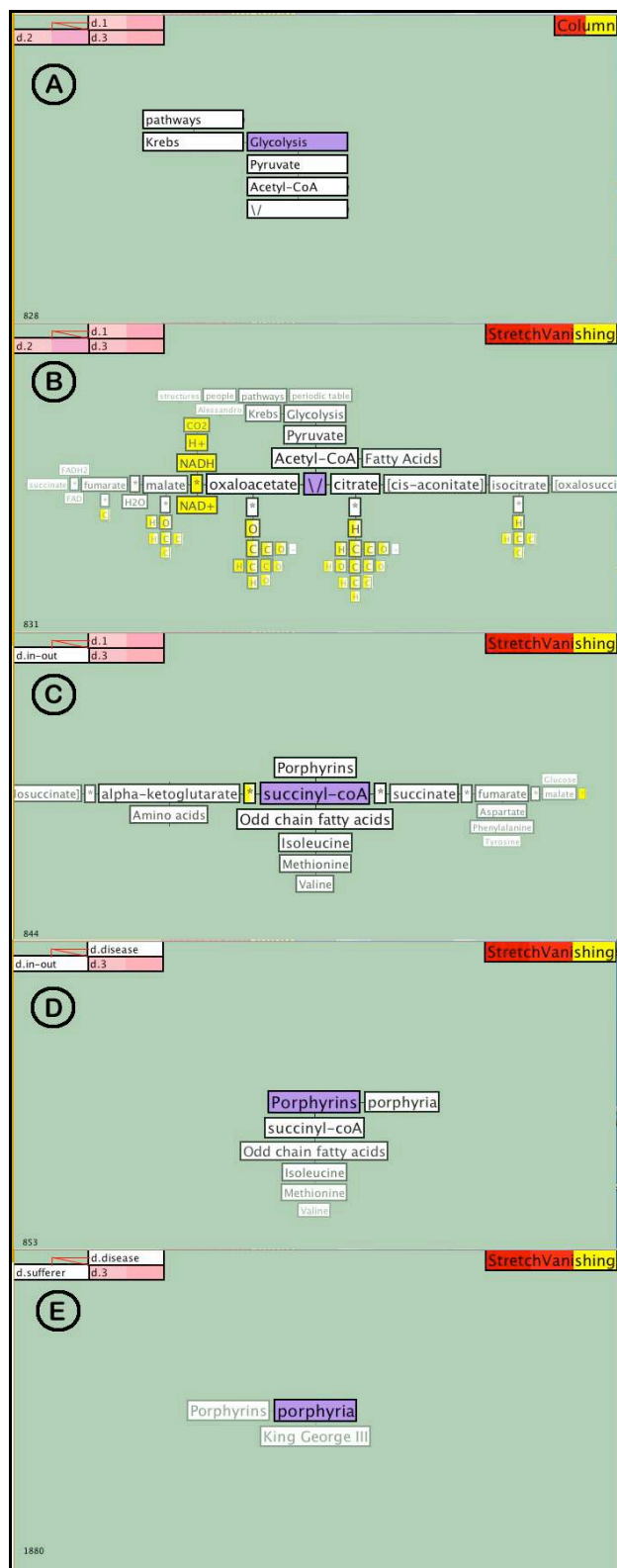


Figure 16. The BioInformatics Workbench: The Krebs Cycle. *A. The Krebs cycle is the chemistry of glycolysis. B. The main cycle. C. d.in-out shows that porphyrins are produced. D. d.disease shows that the porphyrins are involved in porphyria. E. d.sufferer shows that King George III suffered from porphyria.*

Of course, the above examples are more suited to explanations of simple BioInformatics and education by exploration, rather than the more complex challenges posed by proteomics, genomics, analyses of microarrays and so on. However, currently, the user interfaces of most bioinformatics tools are either perl scripts or HTML forms. The above examples demonstrate both how a singular information space may be used for the many complex data types that can be found in bioinformatics, and how this may be utilised for the management and analysis of raw data.

7. Conclusions

Although ZigZag has been available for a number of years now, both as a conceptual framework and as experimental software, real-world applications of it have been rare. In this paper we have described two such systems. Both of these are currently at the prototype stage of development, but they are being targeted very firmly at topical, real-world problems. These problems are in totally unrelated areas, and superficially extremely dissimilar. However, the similarities between them are in many respects far more striking than these differences. Both systems work with deeply interconnected data that is difficult to visualize by conventional means. Moreover, both systems are highly amenable to description by the orthogonal interconnections of a zzStructure.

Despite the benefits we have shown, there are a number of limitations of the ZigZag paradigm that will require further investigation – especially with regard to the strict enforcement of data structures and types, and the current interaction models. With regard to the typing of data and enforcement of its structures, both XML and databases have schema languages, which allow the expression of the rules for the data to be entered. In the future it is likely that the need for this will be satisfied by a set of template toolkits, which would enforce the data entry rules. This may need to be an external application, rather than an aptitude, as ZigZag itself enforces the openness of data, however this will require much deeper investigation.

Whilst ZigZag is built upon a foundation of extremely simple rules, as with chess, out of simplicity there arises the potential for great complexity. It is because of this potential for complexity that ZigZag is a truly generalisable system, and one that is highly suited to complex information with deeply interconnected, loosely structured information – such as those we have described here. The key to harnessing this enormous potential is to build zzStructures that conform to Einstein's ideal of being as simple as possible, but no simpler.

8. Acknowledgements

We should like to thank Ted Nelson for inspiration, encouragement and guidance. We are also most grateful to Marlene Mallicoat and the members of the Web Technology Group at the University of Nottingham for many helpful comments and discussions.

The development of zzPhone was supported by Symbian UK; and the development of the Bioinformatics Workspace was funded by the University of Nottingham, during the tenure of Professor Nelson as a Leverhulme Visiting Professor.

9. References

- [1] Lukka, T. & Ervasti, K. GZigZag – a Platform for Cybertext Experiments. In: CyberText Yearbook 2000, Eds Eskelinen, M. & Koskimaa, R. Eastgate Systems, MA. 2000. Available via: <http://gzigzag.sourceforge.net/ct/ct.html>
- [2] McGuffin, M.J., A Graph-Theoretic Introduction to Ted Nelson's Zzstructures, January 2004. Available from: <http://www.dgp.utoronto.ca/~mjmcguff/research/zigzag/>
- [3] Moore, A. & Brailsford, T.J. Unified Hyperstructures for Bioinformatics: escaping the application prison. 2004. Journal of Digital Information: Special Issue on Future Visions of Common-Use: Volume 5 Issue 1, Article No. 254, 2004-05-27. Available from: <http://jodi.ecs.soton.ac.uk/Articles/v05/i01/Moore/>
- [4] Nelson, T.H. Computer Lib/Dream Machines. Published by the author. 1974. Republished by Microsoft Press, 1988.
- [5] Nelson, T.H. What's on my mind? First Wearable Computer Conference, Fairfax VA. 1998. Available via: <http://www.xanadu.com.au/ted/zigzag/xybrap.html>
- [6] Nelson, T.H. ZigZag Tutorial. 1999. Available via: <http://www.xanadu.net/zigzag/tutorial/ZZwelcome.html>
- [7] Nelson, T.H. Deeper cosmology, deeper documents. Technical Briefing, Twelfth ACM Conference on Hypertext and Hypermedia, Århus, Denmark, 2001. Available via: <http://www.ht01.org/tech.html>
- [8] Nelson, T.H. Structure, tradition and possibility, Keynote. Proceedings of the Fourteenth ACM Conference on Hypertext and Hypermedia, Nottingham, UK, 2003, pp 5-6. Available via: <http://www.ht03.org/keynote-ted.html>.
- [9] Nelson, T.H. Cosmology for a different computer universe: data model, mechanisms, virtual machine and visualization infrastructure. 2004. Journal of Digital Information: Special Issue on Future Visions of Common-Use Hypertext. (in press).